

FluxASM / FluxIR Technical Specification

Version 1.1 - July 2026

Author: Jacob (SageOS / SageVM Project)

A minimalist technical specification for a dataflow-oriented assembly language and its canonical machine-readable representation.

FluxASM and FluxIR

Minimalist edition with structured navigation and machine-readable examples.

Table of Contents

Placeholder for table of contents	0
-----------------------------------	---

1. Introduction

FluxASM is a dataflow-oriented assembly language designed for platform-agnostic and hardware-agnostic execution. It models computation as a graph of waves, units, and instructions, enabling deterministic parallelism and explicit control over side effects.

FluxIR is the canonical machine-readable representation of FluxASM. It is designed for:

- Compilers
- JIT engines
- Simulation environments
- Heterogeneous hardware backends

Together, FluxASM and FluxIR define a complete low-level programming model suitable for CPUs, GPUs, DSPs, accelerators, and virtual machines.

2. Execution Model

FluxASM programs execute as a directed acyclic graph (DAG) per wave activation. Waves can be entered multiple times via branches, and wave parameters carry values between iterations, keeping the data dependencies acyclic within each wave body.

2.1 Waves

A wave is a scheduling domain.

- A wave can declare formal parameters (input registers) that receive values when the wave is entered via a branch.
- Units inside a wave may execute in any order subject to their explicit dependency edges.
- Waves define phases, not sequential instruction streams; control transfer between waves is done by branch and goto instructions.

2.2 Units

A unit is the smallest schedulable block.

Each unit declares:

- uses: registers required before execution
- defs: registers produced
- latency: non-semantic timing hint
- resources: abstract resource usage
- predicate: optional execution mask

Units contain instructions that execute with implicit dataflow ordering; an instruction's operands are ready only after all their producers have completed, enforced via unit-level deps.

2.3 Dependencies

Dependencies are explicit:

```
.dep U0 -> U1 (v3)
.dep U1 -> U2 (s0, p0)
```

Backends must respect all dependency edges. This includes intra-wave edges; within a wave, independent units may execute in any order, but a dep forces sequencing.

2.4 Effects

Effects represent deterministic side-effect channels:

- IO devices
- Shared memory
- Atomics
- External systems

Effect registers are tokens that carry ordering and resource identity. They are produced by device binding or ordering instructions and consumed by side-effecting operations. This makes effect ordering part of the dataflow graph.

3. Type System

FluxASM uses a minimal, portable type system.

3.1 Scalar Types

Type	Description
i8, i16, i32, i64	Signed integers
u8, u16, u32, u64	Unsigned integers
f16, f32, f64	Floating-point

3.2 Vector Types

$vN<T>$

- N: number of lanes
- T: scalar type

3.3 Predicate Type

pred - boolean mask.

3.4 Effect Type

effect - opaque handle to a side-effect channel.

4. Register Model

FluxASM defines abstract register classes:

Class	Prefix	Example
Scalar	s	s0
Vector	v	v3
Predicate	p	p0

Effect	e	e0
--------	---	----

Backends decide physical mapping and spilling.

5. Instruction Set

FluxASM defines semantic opcodes. Backends map them to hardware instructions. New instructions for binding and effect ordering are included.

5.1 Arithmetic

```
add.s dst, a, b
sub.s dst, a, b
mul.s dst, a, b
div.s dst, a, b
add.v vdst, va, vb
mul.v vdst, va, vb
reduce_sum.s sdst, vsrc
```

5.2 Logic

```
and.s dst, a, b
or.s dst, a, b
xor.s dst, a, b
not.s dst, src
cmp.s pdst, a, b, OP ; OP is EQ, NE, LT, GT, etc.
sel.s dst, pred, tval, fval
```

5.3 Memory

All memory operations now take an effect register that identifies the memory region and ordering token.

```
ld.s dst, eRegion, [base + offset]
st.s eRegion, [base + offset], src
ld.v vdst, eRegion, [base + offset]
st.v eRegion, [base + offset], vsrc
```

base is a scalar register, offset is an immediate or register. The effect register eRegion must be bound to a memory region (see 6.2).

5.4 Control

Wave parameters make loops first-class. A wave can declare a list of parameters (registers and their types). Branch instructions pass actual values for these parameters.

```
.wave Wloop (s_acc:i32, s_ptr:i32)
branch cond, Wloop(s_newacc, s_nextptr), Wend
goto W_next
end terminates the program. A wave containing end must not contain any branch instruction (the end
replaces branching). Multiple terminal waves are allowed; the first one reached halts execution.
```

5.5 Effect Binding

Devices and memory regions are abstract resources that must be bound to effect registers before use.

```
bind_device eX, DEVICE("log")
bind_memory eX, MEM_REGION("shared0")
```

These instructions create a fresh effect token that identifies the resource. All subsequent IO or memory accesses using that effect token will target that resource.

5.6 Effect Ordering

Effect tokens can be reordered or serialized using barrier/fence instructions that consume an old effect register and produce a new one.

```
e_barrier eX, eY      ; eX old token, eY new token - all prior writes to this channel are visible
                      to later uses via eY.
e_fence   eX, eY, MODE ; MODE is "acquire", "release", or "acq_rel"
For example:
U1: write e0, data
U2: e_fence e0, e1, "release"
U3: read e1, dst
```

5.7 Atomics

Atomics also require an effect register.

```
atomic_add eX, [addr], src
atomic_cmpxchg eX, [addr], expected, new
```

The address `addr` is a scalar register with an optional offset (like loads). The effect register provides both ordering and the memory space.

6. Effects & Memory Model

Effects are deterministic tokens for side-effects. They are created by binding, passed through barriers/fences, and consumed by memory and IO operations.

6.1 Devices

Devices are abstract resources declared in the module and bound to effect registers.

```
.io e0, DEV_GENERIC("log") ; in FluxASM shorthand (expands to bind_device)
```

In FluxIR, a device is declared separately and then referenced in a `bind_device` instruction.

6.2 Memory Regions

Memory regions are abstract spaces. A region is declared and then bound to an effect register, which is used in all loads/stores/atomics to that region.

```
.mem e1, MEM_REGION("shared0") ; expands to bind_memory
```

Backends map regions to:

- Address ranges
- NUMA nodes
- GPU shared memory
- Virtual memory segments

6.3 Ordering Guarantees

Effect registers define a happens-before chain. If two effectful operations use tokens `eA` and `eB` where there exists a path `eA -> ... -> eB` in the dataflow graph, then all operations on `eA` are visible to those on `eB`. This subsumes traditional memory barriers while keeping the graph explicit.

7. FluxIR Canonical Representation

FluxIR is a JSON-like graph representation. The following sections define the updated structure.

7.1 Module Structure

```
{
  "module": {
    "name": "demo",
    "version": "1.1",
    "entry_points": [...],
    "types": [...],
    "registers": [...],
    "devices": [...],
    "memory_regions": [...],
    "waves": [...],
    "units": [...],
    "deps": [...],
    "control_graph": [...]
  }
}
```

7.2 Entry Points

Defines parameters, returns, and starting wave (unchanged).

7.3 Types

Scalar, vector, predicate, and effect types (unchanged).

7.4 Registers

Declares register class and type. Registers used as wave parameters are declared here as usual.

7.5 Devices

Abstract devices for effects. Example:

```
{
  "devices": [
    { "name": "log", "kind": "generic", "properties": { "category": "logging" } }
  ]
}
```

7.6 Memory Regions

Abstract memory spaces.

```
{
  "memory_regions": [
    { "name": "shared0", "kind": "shared", "size": 4096, "attributes": {} }
  ]
}
```

7.7 Waves

Waves now have an optional params array. Each parameter is a register and its type.

```
{
  "name": "wloop",
  "params": [
    { "reg": "s_acc", "type": "i32" },
    { "reg": "s_ptr", "type": "i32" }
  ],
  "units": ["U_body", "U_check"]
}
```

If params is empty or absent, the wave takes no arguments.

7.8 Units

Unchanged, except instructions can now include binding and updated memory/effect opcodes.

7.9 Instructions

All opcodes remain semantic. Operands now include effect registers for memory ops and binding.

Binding instruction example:

```
{
  "opcode": "bind_device",
  "operands": [
    { "kind": "reg", "name": "e_log" },
    { "kind": "ref", "type": "device", "name": "log" }
  ]
}
```

Memory load with effect:

```
{
  "opcode": "ld.s",
  "operands": [
    { "kind": "reg", "name": "s_dst" },
    { "kind": "reg", "name": "e_shared" },
    { "kind": "reg", "name": "s_base" },
    { "kind": "imm", "value": 0 }
  ]
}
```

Branch with arguments:

```
{
  "opcode": "branch",
  "operands": [
    { "kind": "reg", "name": "p_cond" },
    { "kind": "wave", "name": "wloop", "args": {
      "s_acc": "s_newacc",
      "s_index": "s_newindex"
    } },
    { "kind": "wave", "name": "wend", "args": {} }
  ]
}
```

Effect fence:

```
{
  "opcode": "e_fence",
  "operands": [
    { "kind": "reg", "name": "e_new" },
    { "kind": "reg", "name": "e_old" },
    { "kind": "imm", "value": "release" }
  ]
}
```

7.10 Dependencies

Unchanged: explicit edges between units.

7.11 Control Graph

A mandatory list of all allowed control-flow edges between waves. Used for validation.


```
{
  "control_graph": [
    { "from": "w0", "to": ["wloop"] },
    { "from": "wloop", "to": ["wloop", "wend"] }
  ]
}
```

Every branch target must appear in the control graph.

8. Validation Rules

A FluxIR validator must enforce:

8.1 Structural

- All waves referenced must exist.
- All units must belong to exactly one wave.
- All registers must be declared before use.
- Wave parameters must be declared in the registers section and their types must match.

8.2 Type Safety

- Operand types must match opcode requirements.
- Vector/scalar mismatches are illegal.
- Predicates must be pred.
- Effect registers can only be used by operations that expect an effect.

8.3 Dependency Safety

A unit cannot use a register unless it is a parameter of the enclosing wave or is defined by a dependency chain from a producer unit.

For a unit in a wave that has parameters, those parameters are considered available at wave entry; they are not used in the deps because they are satisfied by the wave call.

8.4 Effect Safety

- Every effect register used in an IO or memory operation must be either the output of a `bind_device/bind_memory` instruction or derived from such through `e_barrier/e_fence` chains.
- No effect register may be used before it is defined (single-assignment).

8.5 Control Safety

- All branch targets must be listed in the `control_graph` and must be valid waves.
- A wave that contains an end instruction must have no outgoing branches.
- If a wave contains a branch instruction, it must not contain end.
- A branch to a wave with parameters must provide exactly the matching number and types of arguments.
- `goto` is only allowed to waves with zero parameters.

9. Reference Example

This example computes a sum of an array (starting at address `s_ptr`, length `s_len`) using a loop, then logs the result. It demonstrates wave parameters, device/memory binding, and effect token threading.

9.1 Entry, Types, Registers

```
{
  "module": {
    "name": "array_sum",
    "version": "1.1",
    "entry_points": [
      {
        "name": "main",
        "params": [
          { "reg": "s_ptr", "type": "i32" },
          { "reg": "s_len", "type": "i32" }
        ],
        "returns": [],
        "start_wave": "w0"
      }
    ],
    "types": [
      { "name": "i32", "kind": "scalar", "bits": 32 },
      { "name": "pred", "kind": "predicate" },
      { "name": "effect", "kind": "effect" }
    ],
    "registers": [
      { "name": "s_ptr", "class": "scalar", "type": "i32" },
      { "name": "s_len", "class": "scalar", "type": "i32" },
      { "name": "s_acc", "class": "scalar", "type": "i32" },
      { "name": "s_index", "class": "scalar", "type": "i32" },
      { "name": "s_val", "class": "scalar", "type": "i32" },
      { "name": "s_newacc", "class": "scalar", "type": "i32" },
      { "name": "s_newindex", "class": "scalar", "type": "i32" },
      { "name": "p_cond", "class": "predicate", "type": "pred" },
      { "name": "e_mem", "class": "effect", "type": "effect" },
      { "name": "e_log", "class": "effect", "type": "effect" }
    ]
  }
}
```

9.2 Devices and Memory Regions

```
{
  "devices": [
    { "name": "log", "kind": "generic", "properties": { "category": "logging" } }
  ],
  "memory_regions": [
    { "name": "data_region", "kind": "global", "size": 0, "attributes": {} }
  ]
}
```

9.3 Waves

```
{
  "waves": [
    { "name": "w0", "params": [], "units": ["U_bind"] },
    { "name": "wloop", "params": [
      { "reg": "s_acc", "type": "i32" },
      { "reg": "s_index", "type": "i32" }
    ],
      "units": ["U_body", "U_check"]
    },
    { "name": "wend", "params": [], "units": ["U_log"] }
  ]
}
```

9.4 Units and Dependencies

```

{
  "units": [
    {
      "name": "U_bind",
      "wave": "w0",
      "uses": ["s_ptr", "s_len"],
      "defs": ["e_mem", "e_log", "s_acc", "s_index"],
      "instructions": [
        {
          "opcode": "bind_memory",
          "operands": [
            { "kind": "reg", "name": "e_mem" },
            { "kind": "ref", "type": "memory_region", "name": "data_region" }
          ]
        },
        {
          "opcode": "bind_device",
          "operands": [
            { "kind": "reg", "name": "e_log" },
            { "kind": "ref", "type": "device", "name": "log" }
          ]
        },
        {
          "opcode": "mov.s",
          "operands": [
            { "kind": "reg", "name": "s_acc" },
            { "kind": "imm", "value": 0 }
          ]
        },
        {
          "opcode": "mov.s",
          "operands": [
            { "kind": "reg", "name": "s_index" },
            { "kind": "imm", "value": 0 }
          ]
        }
      ]
    }
  ]
}

{
  "units": [
    {
      "name": "U_body",
      "wave": "wloop",
      "uses": ["s_acc", "s_index", "e_mem", "s_ptr"],
      "defs": ["s_val", "s_newacc", "s_newindex"],
      "instructions": [
        {
          "opcode": "ld.s",
          "operands": [
            { "kind": "reg", "name": "s_val" },
            { "kind": "reg", "name": "e_mem" },
            { "kind": "reg", "name": "s_ptr" },
            { "kind": "reg", "name": "s_index" }
          ]
        },
        {
          "opcode": "add.s",
          "operands": [
            { "kind": "reg", "name": "s_newacc" },
            { "kind": "reg", "name": "s_acc" },
            { "kind": "reg", "name": "s_val" }
          ]
        }
      ]
    }
  ]
}

```

```

        {
            "opcode": "add.s",
            "operands": [
                { "kind": "reg", "name": "s_newindex" },
                { "kind": "reg", "name": "s_index" },
                { "kind": "imm", "value": 1 }
            ]
        }
    ]
}
]
}
{
    "units": [
        {
            "name": "U_check",
            "wave": "wloop",
            "uses": ["s_newindex", "s_len"],
            "defs": ["p_cond"],
            "instructions": [
                {
                    "opcode": "cmp.s",
                    "operands": [
                        { "kind": "reg", "name": "p_cond" },
                        { "kind": "reg", "name": "s_newindex" },
                        { "kind": "reg", "name": "s_len" },
                        { "kind": "imm", "value": "LT" }
                    ]
                }
            ]
        },
        {
            "name": "U_branch",
            "wave": "wloop",
            "uses": ["p_cond", "s_newacc", "s_newindex"],
            "defs": [],
            "instructions": [
                {
                    "opcode": "branch",
                    "operands": [
                        { "kind": "reg", "name": "p_cond" },
                        { "kind": "wave", "name": "wloop", "args": {
                            "s_acc": "s_newacc",
                            "s_index": "s_newindex"
                        } },
                        { "kind": "wave", "name": "wend", "args": {} }
                    ]
                }
            ]
        },
        {
            "name": "U_log",
            "wave": "wend",
            "uses": ["s_acc", "e_log"],
            "defs": [],
            "instructions": [
                {
                    "opcode": "write",
                    "operands": [
                        { "kind": "reg", "name": "e_log" },
                        { "kind": "reg", "name": "s_acc" }
                    ]
                }
            ]
        },
        {
            "opcode": "end",

```

```

        "operands": []
      }
    ]
  }
]
}
{
  "deps": [
    { "from": "U_bind", "to": "U_body", "regs": ["e_mem", "s_ptr", "s_len"] },
    { "from": "U_body", "to": "U_check", "regs": ["s_newindex"] },
    { "from": "U_body", "to": "U_branch", "regs": ["s_newacc", "s_newindex"] },
    { "from": "U_check", "to": "U_branch", "regs": ["p_cond"] }
  ],
  "control_graph": [
    { "from": "w0", "to": ["wloop"] },
    { "from": "wloop", "to": ["wloop", "wend"] }
  ]
}

```

10. Glossary

Term	Definition
Wave	Scheduling domain containing units. May have parameters.
Unit	Smallest schedulable block.
Effect	Deterministic side-effect channel token.
FluxASM	Human-readable assembly language.
FluxIR	Canonical machine-readable representation.
Device	Abstract IO or system interface.
Memory Region	Abstract memory space.
Wave Parameter	Register bound on entry to a wave, enabling loop-carried values.

End of specification.